1 React Phase 2 – useEffect, Cleanup Function, Timer, API Fetching & Memory Leak

2 Introduction

React ma Component, Props, State, ra Event Handling sikepaxi, next important concept **Side Effects** ho. Real-world application haru ma data server bata lyaunu, timer chalau, localStorage use garnu, browser title change garnu, event listener add garnu jasta kaam haru garna parxa. Yei sabai kaam React ma `useEffect` Hook le handle garxa.

---

3 1. Side Effect

4 Side Effect Vaneko K Ho?

React component ko main responsibility UI render garnu ho.

Example:

```
function App() {
    return <h1>Hello React</h1>;
}
```

Yo component le screen ma matra UI dekhauxa.

Tara real project ma UI matra hudaina.

Examples of Side Effects:

- API bata data fetch garnu
- Timer (`setInterval`) chalau
- Browser title change garnu
- localStorage ma data save/read garnu
- Event Listener add/remove garnu
- WebSocket connection banaunu

Simple Definition:

React ko rendering process bahira hune kaam lai Side Effect vaninxa.

---

5 2. useEffect Hook

`useEffect` React le Side Effects handle garna diyeko Hook ho.

Syntax:

```
useEffect(() => {

});
```

Example:

```
useEffect(() => {
    console.log("Component Loaded");
});
```

Yo component render huda function execute hunxa.

---

6 3. Dependency Array

`useEffect` ko second parameter Dependency Array ho.

Syntax:

```
useEffect(() => {

}, []);
```

Dependency Array le React lai effect kahile run garne vanera decide garna help garxa.

---

7 Case 1: No Dependency Array

```
useEffect(() => {
    console.log("Hello");
});
```

Result:

- Initial render ma run hunxa.
- Harek re-render ma feri run hunxa.

Example:

```
Load
Click
Click
Click
```

Output:

```
Hello
Hello
Hello
Hello
```

---

8 Case 2: Empty Dependency Array

```
useEffect(() => {
    console.log("Component Mounted");
}, []);
```

Result:

- Component mount huda ek choti matra run hunxa.

Example:

```
Page Load
```

Output:

```
Component Mounted
```

Button click gare pani feri run hudaina.

---

9 Mount Vaneko K Ho?

Mount = Component first time screen ma dekhinu.

Flow:

```
Application Start
        ↓
Component Create
        ↓
Component Screen Ma Dekhiyo
        ↓
Mounted
```

---

10 Case 3: Specific Dependency

```
useEffect(() => {
    console.log("Count Changed");
}, [count]);
```

Result:

- `count` change huda matra effect run hunxa.

Example:

```
const [count, setCount] = useState(0);
const [name, setName] = useState("");
```

- Count change → Effect run hunxa.
- Name change → Effect run hudaina.

---

11 Dependency Array Summary

Dependency	Effect Kahile Run Hunxa?
No Dependency	Every Render
[]	Only Once (Mount)
[count]	Count change huda
[name]	Name change huda
[count, name]	Count wa Name change huda

---

12 4. Cleanup Function

Cleanup Function useEffect vitra return garinxa.

Syntax:

```
useEffect(() => {

    return () => {

    };

}, []);
```

Yo function component unmount huda execute hunxa.

---

13 Unmount Vaneko K Ho?

Unmount = Component screen bata remove huda.

Flow:

```
Component Mounted
        ↓
User Uses Component
        ↓
Component Removed
        ↓
Unmount
```

---

14 Example

```
useEffect(() => {

    console.log("Mounted");

    return () => {
        console.log("Unmounted");
    };

}, []);
```

Output:

Page Load

```
Mounted
```

Page Leave

```
Unmounted
```

---

15 Cleanup Function Kina Chainxa?

Real-world ma timer, event listener, WebSocket, API request jasta resources cleanup nagare memory leak huna sakxa.

Cleanup Function le unnecessary resources remove garxa.

---

16 5. Timer (setInterval)

JavaScript ko built-in function ho.

Syntax:

```
setInterval(callback, milliseconds);
```

Example:

```
setInterval(() => {
    console.log("Hello");
}, 1000);
```

Output:

```
Hello
Hello
Hello
...
```

1000 milliseconds = 1 second.

---

17 React Timer Example

Wrong:

```
useEffect(() => {

    setInterval(() => {
        console.log("Tick");
    }, 1000);

}, []);
```

Problem:

Timer page leave garepaxi pani chalirahanxa.

Correct:

```
useEffect(() => {

    const timer = setInterval(() => {
        console.log("Tick");
    }, 1000);

    return () => {
        clearInterval(timer);
    };

}, []);
```

---

18 Live Counter Example

```
import { useState, useEffect } from "react";

function App() {

    const [count, setCount] = useState(0);

    useEffect(() => {

        const timer = setInterval(() => {
            setCount(prev => prev + 1);
        }, 1000);

        return () => {
            clearInterval(timer);
        };

    }, []);

    return <h1>{count}</h1>;
}
```

Output:

```
0
1
2
3
4
5
...
```

---

19 Why Functional Update?

Preferred:

```
setCount(prev => prev + 1);
```

Instead of:

```
setCount(count + 1);
```

Reason:

`setInterval`, `setTimeout`, ra asynchronous code ma callback le purano state (`stale state`) samatna sakxa.

Functional Update le hamesha latest state use garxa.

---

20 6. API Fetching

API (Application Programming Interface) server bata data receive garne medium ho.

Flow:

```
React
    ↓
API Request
    ↓
Server
    ↓
JSON Data
    ↓
React UI
```

---

21 Example

```
useEffect(() => {

    fetch("https://jsonplaceholder.typicode.com/users")
        .then(response => response.json())
        .then(data => {
            console.log(data);
        });

}, []);
```

---

22 API + useState

```
import { useState, useEffect } from "react";

function App() {

    const [users, setUsers] = useState([]);

    useEffect(() => {

        fetch("https://jsonplaceholder.typicode.com/users")
            .then(res => res.json())
            .then(data => setUsers(data));

    }, []);

    return (
        <div>
            {
                users.map(user => (
                    <p key={user.id}>{user.name}</p>
                ))
            }
        </div>
    );
}
```

---

23 API Fetch Garda [] Kina Use Garinxa?

Correct:

```
useEffect(() => {

    fetch(...);

}, []);
```

Reason:

Page load huda ek choti matra API call hos.

Wrong:

```
useEffect(() => {

    fetch(...);
```

```
});
```

Result:

- Render
- API Call
- State Update
- Re-render
- API Call Again

Infinite Loop huna sakxa.

---

24 7. Memory Leak

Memory Leak = Use gareko memory release nagarnu.

Wrong Example:

```
useEffect(() => {

    setInterval(() => {
        console.log("Running");
    }, 1000);

}, []);
```

Problem:

Component remove bhayepaxi pani timer chalirahanxa.

Correct:

```
useEffect(() => {

    const timer = setInterval(() => {
        console.log("Running");
    }, 1000);

    return () => {
        clearInterval(timer);
    };

}, []);
```

---

25 8. Event Listener Cleanup

Wrong:

```
useEffect(() => {

    window.addEventListener("resize", handleResize);

}, []);
```

Correct:

```
useEffect(() => {

    window.addEventListener("resize", handleResize);

    return () => {
        window.removeEventListener("resize", handleResize);
    };

}, []);
```

---

26 9. WebSocket Cleanup

```
useEffect(() => {

    const socket = new WebSocket(url);

    return () => {
        socket.close();
    };

}, []);
```

---

27 Complete Life Cycle

```
Application Start
        ↓
Component Mounted
        ↓
useEffect Runs
        ↓
Timer/API/Event Listener Starts
        ↓
User Uses Component
        ↓
Component Removed
        ↓
```

```
Cleanup Function Executes
        ↓
Resources Released
```

---

28 Interview Questions

29 1. useEffect kina use garinxa?

Side Effects handle garna, jastai API Fetching, Timer, Event Listener, Local Storage, Browser Title Update, WebSocket, etc.

---

30 2. Cleanup Function kahile run hunxa?

- Component unmount huda.
- Dependency change huda purano effect cleanup garna.

---

31 3. clearInterval() kina use garinxa?

Running timer stop garna ra Memory Leak avoid garna.

---

32 4. API Fetch garda [] kina use garinxa?

API page load huda ek choti matra call hos vanera.

---

33 5. Memory Leak Vaneko K Ho?

Component remove bhayepaxi pani timer, event listener, socket jasta resources background ma chalirahaney situation lai Memory Leak vaninxa.

---

34 Key Takeaways

- `useEffect` Side Effects handle garna use garinxa.
- Dependency Array le effect kahile run garne decide garxa.
- `[]` use gare effect mount huda ek choti matra run hunxa.

- Cleanup Function (`return () => {}`) resources release garna use garinxa.
- Timer use garda `clearInterval()` garna birsinu hudaina.
- API Fetch garda `useEffect([])` best practice ho.
- Functional Update (`prev => prev + 1`) asynchronous updates ma safe approach ho.
- Memory Leak avoid garna timer, event listener, WebSocket sabai cleanup garna parxa.